EWAS Power Calculations (Placenta WGBS)

George Kuodza

2026-03-03 at 16:11

Introduction

EpipwR is a quasi-simulation based approach to power analysis for epigenome-wide association studies (EWAS) with a continuous or binary outcome variable. It is fast, efficient and allows the exact level of precision to be set by the user. EpipwR is the first open source tool able to calculate power for EWAS with a continuous outcome. For binary outcomes, it re-implements the pwrEWAS methodology (Graw et al., 2019 - currently available on GitHub) with a few changes and significantly improves the computation time. EpipwR is simple in that it has three main functions: `get_power_cont()` and `get_power_cc` calculate power for various settings with continuous and binary outcomes (respectively) and `EpipwR_plot()` plots the output. The following vignette describes in detail the EpipwR workflow for an analysis of a continuous outcome.

Methodology (Continuous Outcome)

Suppose an upcoming EWAS plans to analyze the methylation levels at K CpG sites and some continuous outcome Y. Of the K CpG sites, it is assumed that only a subest of these (K_m) have non-zero correlation to Y. The goal of EpipwR is to determine the level of power achieved by this study at a given sample size n, where power is defined as the proportion of the K_m CpG sites are found to have a statistically significant association with Y.

To begin, EpipwR generates data for the K_m CpG methylation levels and for Y. Users specify μ_ρ and σ_ρ, the average and standard deviation of the correlation between each of the K_m methylation levels and Y. If users prefer that all K_m methylation levels have equal correlation with Y, then σ_ρ can be set to 0. Like many power analysis tools, EpipwR assumes no correlation across CpG sites (i.e., they are conditionally independent given Y). Users also select which empirical EWAS data to use to generate initial methylation levels. Full details of the data generation process can be found in the corresponding EpipwR article (Barth and Reynolds, 2024).

Once a dataset has been generated, p-values are calculated by performing the specified hypothesis test (Pearson, Spearman or Kendall correlation) for each K_m site. Please note that in absence of any "vertical" correlation or additional covariates, performing a standard analysis in limma is equivalent to using the Pearson correlation test. Users can specify either a false discovery rate or family-wise type I error rate correction for determining statistical significance. If the false discovery rate is selected, p-values for the remaining null tests are generated via a $Uniform(0, 1)$ distribution.

Since the process described above is based heavily on random sampling, it must be repeated on several data sets to produce a reliable power estimate. Whereas traditional tools require the user to specify the number of repetitions, EpipwR sets 20 to the minimum number of data sets, allows users to set a maximum number of data sets and a level of precision in the form of a 95% confidence interval margin of error. Anytime after the first 20 datasets, EpipwR terminates once one of these two conditions are met. The default margin of error is 3% (it is recommended that this be between 1 and 5%). For example, if the target margin of error is 3% and the maximum number of data sets is 1000, then the algorithm stops when $N = 1000$ or once the following conditions are met:

$$1.96 \cdot \frac{s_{(N)}}{\sqrt{N}} < .03, \qquad N \geq 20$$

where $s_{(N)}$ is the standard deviation of average power over N data sets.

Function Inputs

The below table describes the details of the `get_power_cont` inputs.

Parameter	Description
`dm`	The total number of non-null tests (i.e., the number of CpG sites with methylation having a non-zero correlation with the phenotype).
`Total`	The total number of tests (i.e., total number of CpG sites in the study).
`n`	The sample size of the study - accepts a vector of values.
`fdr_fwer`	Either the false discovery rate (FDR) or the family-wise type I error rate (FWER), depending on `use_fdr.`
`rho_mu`	The average correlation of the non-null tests - accepts a vector of values.
`rho_sd=0`	The standard deviation of the correlation for non-null tests.
`Tissue="Saliva"`	Tissue corresponding to empirical EWAS data (see `?get_power_cont` for valid options).
`Nmax=1000`	The maximum number of data sets that EpipwR will use to determine power. Cannot be less than 20.
`MOE=.03`	The target margin of error of a 95% confidence interval for average power.
`test="Pearson"`	Type of correlation/test utilized. Acceptable inputs are `"Pearson"`, `"Kendall"` and `"Spearman"` .
`use_fdr=T`	Indicates that the false discovery rate should be used as the correction. Otherwise, uses FWER.
`Suppress_Updates=F`	If `T` , removes intermediate status updates.

Methodology (Binary outcome)

For the case-control setting, EpipwR re-implements the pwrEWAS methodology (Graw et al., 2019) with a few key changes to improve speed and interpretation, namely (1) keeping the precision equal between groups rather than variance, (2) requiring users to specify mean and variance for an effect size distribution (rather than a target maximal difference) and (3) generating p-values directly for null tests. Like in `get_power_cont`, `get_power_cc` calculates the optimal amount of data sets needed to determine power at a user-specified level of precision.

EpipwR Workflow specifically for the ECHO project

#ECHO Power Calculation

Power calculations were conducted using the **EpipwR** package under the following assumptions:

- Sample size: **n = 900, 1000, 1100**
- Tissue: **Placenta**
- dm (true non-null CpGs/regions): **1000**
- Error control: **FDR = 0.05**
- CpG universe: **20M and 15M**
- Region universe: **500K and 100K**

- Continuous exposure: **correlation-based effect sizes (Pearson's coefficient) evaluated over a grid of rho_mu values**
- Binary exposure: **25/75 split and 50/50 split**
- Target power: **80%**

```r
library(EpipwR)
library(dplyr)
library(purrr)
library(readr)
library(tibble)
library(kableExtra)
# #some of these package necessary for Rendering
# tinytex::reinstall_tinytex(repository = "illinois")
# tinytex::tlmgr_install("multirow")
# Inputs
n <- c(900, 1000, 1100)
Tissue <- "Placenta"
emp_data <- NULL

dm_values <- c(1000)

Totals <- list(
  CpG_20M     = 2e7,
  CpG_15M     = 1.5e7,
  Region_500K = 5e5,
  Region_100K = 1e5
)

rho_grid   <- c(0.05, 0.10, 0.15, 0.18, 0.20)
delta_grid <- c(0.01, 0.02, 0.03, 0.05)

use_fdr <- TRUE
fdr_fwer <- 0.05

n1_props <- c(`25_75` = 0.25, `50_50` = 0.50)

target_power <- 0.80

out_dir <- "EpipwR_power_rerun_outputs"
if (!dir.exists(out_dir)) dir.create(out_dir, recursive = TRUE)

target_power <- 0.80
```

```r
extract_mdes <- function(df, effect_col, power_col = "avg_power", target_power = 0.80) {

  if (!effect_col %in% names(df)) stop("Missing effect column: ", effect_col)
  if (!power_col %in% names(df)) stop("Missing power column: ", power_col)

  df_sorted <- df %>% arrange(.data[[effect_col]])

  hit <- df_sorted %>%
```

```r
    filter(.data[[power_col]] >= target_power) %>%
    slice(1)

  if (nrow(hit) == 0) {
    max_row <- df_sorted %>% slice(n())
    tibble(
      target_power = target_power,
      mdes_effect = NA_real_,
      achieved_power_at_mdes = NA_real_,
      max_tested_effect = max_row[[effect_col]],
      achieved_power_at_max = max_row[[power_col]]
    )
  } else {
    tibble(
      target_power = target_power,
      mdes_effect = hit[[effect_col]][1],
      achieved_power_at_mdes = hit[[power_col]][1],
      max_tested_effect = max(df_sorted[[effect_col]]),
      achieved_power_at_max = df_sorted %>% slice(n()) %>% pull(.data[[power_col]])
    )
  }
}
```

```r
# --------------------------
# 1) Region continuous (rho)
# --------------------------
out_cont_Region <- map_dfr(dm_values, function(dm) {
  map_dfr(c("Region_500K", "Region_100K"), function(total_name) {
    Total <- Totals[[total_name]]

    get_power_cont(
      dm = dm, Total = Total, n = n,
      fdr_fwer = fdr_fwer,
      rho_mu = rho_grid, rho_sd = 0,
      Tissue = Tissue, emp_data = emp_data,
      use_fdr = use_fdr, test = "pearson"
    ) %>%
      mutate(
        Level = "Region",
        Model = "Continuous (rho; Pearson correlation)",
        Total_name = total_name,
        Total_tests = Total,
        dm = dm
      )
  })
})
head(out_cont_Region)
```

```
##   sample_size rho_mu avg_power     sd_power  N     se_power  Level
## 1         900   0.05   0.00015 0.0003663475 20 8.191780e-05 Region
## 2        1000   0.05   0.00010 0.0003077935 20 6.882472e-05 Region
## 3        1100   0.05   0.00025 0.0004442617 20 9.933993e-05 Region
## 4         900   0.10   0.06570 0.0270887430 20 6.057227e-03 Region
```

```
## 5        1000   0.10   0.10675 0.0223839110 20 5.005195e-03 Region
## 6        1100   0.10   0.15845 0.0222367051 20 4.972278e-03 Region
##                                   Model  Total_name Total_tests   dm
## 1 Continuous (rho; Pearson correlation) Region_500K       5e+05 1000
## 2 Continuous (rho; Pearson correlation) Region_500K       5e+05 1000
## 3 Continuous (rho; Pearson correlation) Region_500K       5e+05 1000
## 4 Continuous (rho; Pearson correlation) Region_500K       5e+05 1000
## 5 Continuous (rho; Pearson correlation) Region_500K       5e+05 1000
## 6 Continuous (rho; Pearson correlation) Region_500K       5e+05 1000
```

```r
mdes_cont_Region <- out_cont_Region %>%
  group_by(sample_size, dm, Total_name, Total_tests, Level, Model) %>%
  group_modify(~ extract_mdes(.x, effect_col = "rho_mu", target_power = target_power)) %>%
  ungroup()

# --------------------------
# 2) Region binary (delta)
# --------------------------

out_bin_Region <- map_dfr(dm_values, function(dm) {
  map_dfr(c("Region_500K", "Region_100K"), function(total_name) {
    Total <- Totals[[total_name]]

    map_dfr(names(n1_props), function(split_name) {
      n1_prop <- n1_props[[split_name]]

      get_power_cc(
        dm = dm, Total = Total, n = n,
        fdr_fwer = fdr_fwer,
        delta_mu = delta_grid, delta_sd = 0,
        n1_prop = n1_prop,
        Tissue = Tissue, emp_data = emp_data,
        use_fdr = use_fdr, test = "pooled"
      ) %>%
        mutate(
          Level = "Region",
          Model = paste0("Binary (delta; ", split_name, ")"),
          Total_name = total_name,
          Total_tests = Total,
          dm = dm,
          n1_prop = n1_prop
        )
    })
  })
})

head(out_bin_Region)
```

```
##   sample_size delta_mu avg_power    sd_power  N    se_power  Level
## 1         900     0.01   0.55825 0.015134050 20 0.003384076 Region
## 2        1000     0.01   0.58115 0.012478508 20 0.002790279 Region
## 3        1100     0.01   0.60490 0.014549010 20 0.003253257 Region
## 4         900     0.02   0.85905 0.010806796 20 0.002416473 Region
## 5        1000     0.02   0.87150 0.009832813 20 0.002198684 Region
```

```
## 6        1100     0.02   0.88345 0.010200490 20 0.002280899 Region
##                    Model  Total_name Total_tests   dm n1_prop
## 1 Binary (delta; 25_75) Region_500K       5e+05 1000    0.25
## 2 Binary (delta; 25_75) Region_500K       5e+05 1000    0.25
## 3 Binary (delta; 25_75) Region_500K       5e+05 1000    0.25
## 4 Binary (delta; 25_75) Region_500K       5e+05 1000    0.25
## 5 Binary (delta; 25_75) Region_500K       5e+05 1000    0.25
## 6 Binary (delta; 25_75) Region_500K       5e+05 1000    0.25
```

```
mdes_bin_Region <- out_bin_Region %>%
  group_by(sample_size, dm, Total_name, Total_tests, Level, Model, n1_prop) %>%
  group_modify(~ extract_mdes(.x, effect_col = "delta_mu", target_power = target_power)) %>%
  ungroup()

# --------------------------
# 3) CpG continuous (rho)
# --------------------------
out_cont_CpG <- map_dfr(dm_values, function(dm) {
  map_dfr(c("CpG_20M", "CpG_15M"), function(total_name) {
    Total <- Totals[[total_name]]

    get_power_cont(
      dm = dm, Total = Total, n = n,
      fdr_fwer = fdr_fwer,
      rho_mu = rho_grid, rho_sd = 0,
      Tissue = Tissue, emp_data = emp_data,
      use_fdr = use_fdr, test = "pearson"
    ) %>%
      mutate(
        Level = "CpG",
        Model = "Continuous (rho; Pearson correlation)",
        Total_name = total_name,
        Total_tests = Total,
        dm = dm
      )
  })
})
head(out_cont_CpG)
```

```
##   sample_size rho_mu avg_power     sd_power  N     se_power Level
## 1         900   0.05   0.00020 0.0005231484 20 1.169795e-04   CpG
## 2        1000   0.05   0.00005 0.0002236068 20 5.000000e-05   CpG
## 3        1100   0.05   0.00010 0.0003077935 20 6.882472e-05   CpG
## 4         900   0.10   0.00280 0.0019084301 20 4.267379e-04   CpG
## 5        1000   0.10   0.00645 0.0053555186 20 1.197530e-03   CpG
## 6        1100   0.10   0.01710 0.0093352755 20 2.087431e-03   CpG
##                                   Model Total_name Total_tests   dm
## 1 Continuous (rho; Pearson correlation)    CpG_20M       2e+07 1000
## 2 Continuous (rho; Pearson correlation)    CpG_20M       2e+07 1000
## 3 Continuous (rho; Pearson correlation)    CpG_20M       2e+07 1000
## 4 Continuous (rho; Pearson correlation)    CpG_20M       2e+07 1000
## 5 Continuous (rho; Pearson correlation)    CpG_20M       2e+07 1000
## 6 Continuous (rho; Pearson correlation)    CpG_20M       2e+07 1000
```

```r
mdes_cont_CpG <- out_cont_CpG %>%
  group_by(sample_size, dm, Total_name, Total_tests, Level, Model) %>%
  group_modify(~ extract_mdes(.x, effect_col = "rho_mu", target_power = target_power)) %>%
  ungroup()

# --------------------------
# 4) CpG binary (delta)
# --------------------------
out_bin_CpG <- map_dfr(dm_values, function(dm) {
  map_dfr(c("CpG_20M", "CpG_15M"), function(total_name) {
    Total <- Totals[[total_name]]

    map_dfr(names(n1_props), function(split_name) {
      n1_prop <- n1_props[[split_name]]

      get_power_cc(
        dm = dm, Total = Total, n = n,
        fdr_fwer = fdr_fwer,
        delta_mu = delta_grid, delta_sd = 0,
        n1_prop = n1_prop,
        Tissue = Tissue, emp_data = emp_data,
        use_fdr = use_fdr, test = "pooled"
      ) %>%
        mutate(
          Level = "CpG",
          Model = paste0("Binary (delta; ", split_name, ")"),
          Total_name = total_name,
          Total_tests = Total,
          dm = dm,
          n1_prop = n1_prop
        )
    })
  })
})

head(out_bin_CpG)
```

```
##   sample_size delta_mu avg_power   sd_power  N    se_power Level
## 1         900     0.01   0.46170 0.01371553 20 0.003066886   CpG
## 2        1000     0.01   0.48700 0.01747479 20 0.003907483   CpG
## 3        1100     0.01   0.51065 0.01951322 20 0.004363289   CpG
## 4         900     0.02   0.79310 0.01463917 20 0.003273418   CpG
## 5        1000     0.02   0.81525 0.01128518 20 0.002523443   CpG
## 6        1100     0.02   0.82390 0.01439262 20 0.003218287   CpG
##                   Model Total_name Total_tests   dm n1_prop
## 1 Binary (delta; 25_75)    CpG_20M       2e+07 1000    0.25
## 2 Binary (delta; 25_75)    CpG_20M       2e+07 1000    0.25
## 3 Binary (delta; 25_75)    CpG_20M       2e+07 1000    0.25
## 4 Binary (delta; 25_75)    CpG_20M       2e+07 1000    0.25
## 5 Binary (delta; 25_75)    CpG_20M       2e+07 1000    0.25
## 6 Binary (delta; 25_75)    CpG_20M       2e+07 1000    0.25
```

```r
mdes_bin_CpG <- out_bin_CpG %>%
  group_by(sample_size, dm, Total_name, Total_tests, Level, Model, n1_prop) %>%
  group_modify(~ extract_mdes(.x, effect_col = "delta_mu", target_power = target_power)) %>%
  ungroup()


full_results <- bind_rows(out_cont_CpG, out_bin_CpG, out_cont_Region, out_bin_Region)

mdes_results <- bind_rows(mdes_cont_CpG, mdes_bin_CpG, mdes_cont_Region, mdes_bin_Region) %>%
  arrange(Level, Total_tests, Model, sample_size, dm)

# # Print MDES summary in the knitted doc
# mdes_results
```

```r
mdes_clean <- mdes_results %>%
  select(Level, Total_name, sample_size, dm, Model,
         mdes_effect, achieved_power_at_mdes)

knitr::kable(
  mdes_clean,
  caption = "Minimum Detectable Effect Size (MDES) summary (target power = 0.80)",
  booktabs = TRUE,
  longtable = TRUE
) |>
  kableExtra::kable_styling(
    latex_options = "repeat_header",
    font_size = 7
  )
```

Table 2: Minimum Detectable Effect Size (MDES) summary (target power = 0.80)

Level	Total_name	sample_size	dm	Model	mdes_effect	achieved_power_at_mdes
CpG	CpG_15M	900	1000	Binary (delta; 25_75)	0.03	0.91670
CpG	CpG_15M	1000	1000	Binary (delta; 25_75)	0.02	0.81215
CpG	CpG_15M	1100	1000	Binary (delta; 25_75)	0.02	0.83165
CpG	CpG_15M	900	1000	Binary (delta; 50_50)	0.02	0.84590
CpG	CpG_15M	1000	1000	Binary (delta; 50_50)	0.02	0.86215
CpG	CpG_15M	1100	1000	Binary (delta; 50_50)	0.02	0.87700
CpG	CpG_15M	900	1000	Continuous (rho; Pearson correlation)	0.20	0.91460
CpG	CpG_15M	1000	1000	Continuous (rho; Pearson correlation)	0.18	0.85570
CpG	CpG_15M	1100	1000	Continuous (rho; Pearson correlation)	0.18	0.91190
CpG	CpG_20M	900	1000	Binary (delta; 25_75)	0.03	0.91365
CpG	CpG_20M	1000	1000	Binary (delta; 25_75)	0.02	0.81525
CpG	CpG_20M	1100	1000	Binary (delta; 25_75)	0.02	0.82390
CpG	CpG_20M	900	1000	Binary (delta; 50_50)	0.02	0.84075
CpG	CpG_20M	1000	1000	Binary (delta; 50_50)	0.02	0.85865
CpG	CpG_20M	1100	1000	Binary (delta; 50_50)	0.02	0.87085
CpG	CpG_20M	900	1000	Continuous (rho; Pearson correlation)	0.20	0.91050
CpG	CpG_20M	1000	1000	Continuous (rho; Pearson correlation)	0.18	0.84675
CpG	CpG_20M	1100	1000	Continuous (rho; Pearson correlation)	0.18	0.90900
Region	Region_100K	900	1000	Binary (delta; 25_75)	0.02	0.88635
Region	Region_100K	1000	1000	Binary (delta; 25_75)	0.02	0.90230
Region	Region_100K	1100	1000	Binary (delta; 25_75)	0.02	0.90850
Region	Region_100K	900	1000	Binary (delta; 50_50)	0.02	0.92035

Table 2: Minimum Detectable Effect Size (MDES) summary (target power = 0.80) *(continued)*

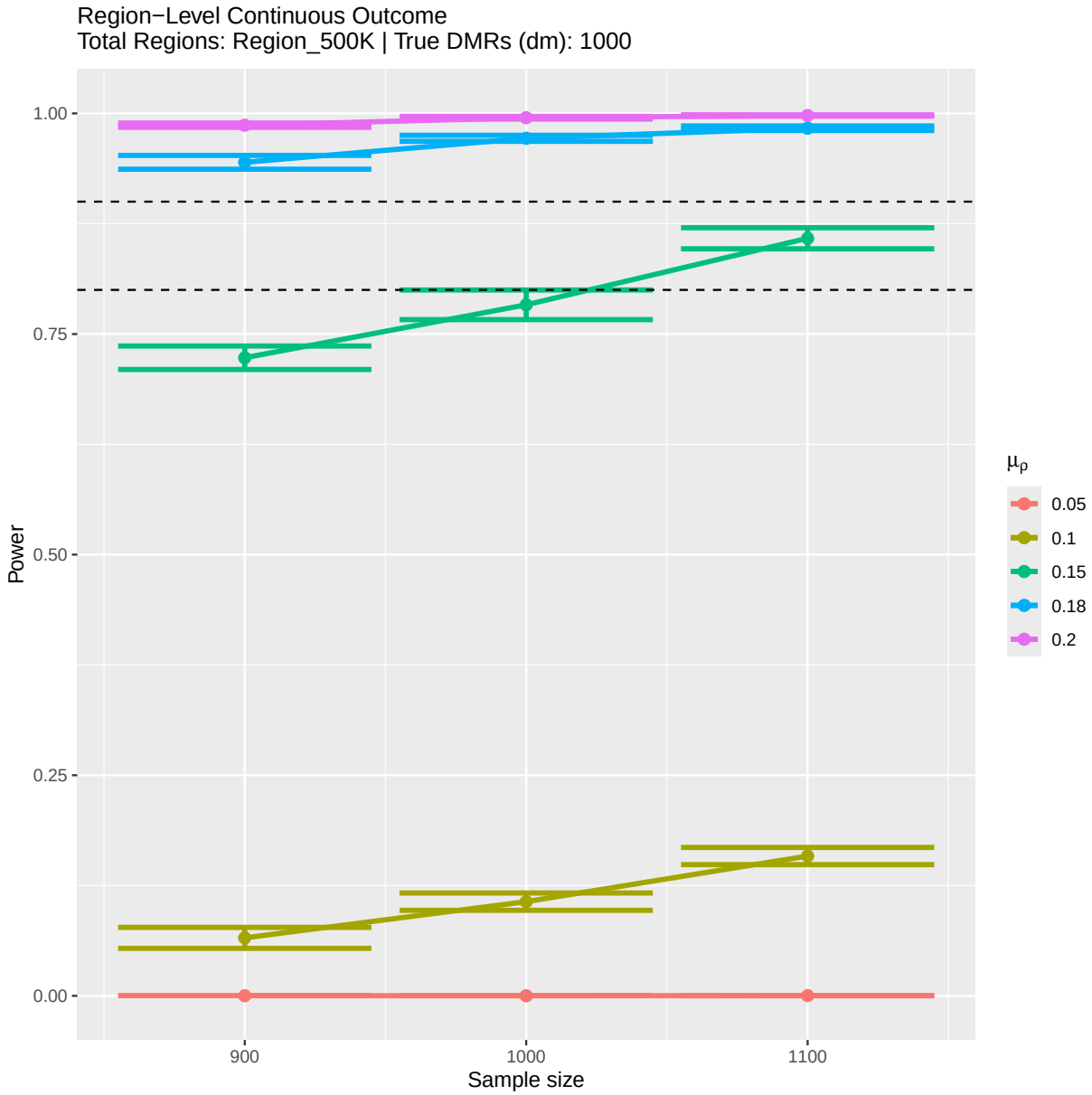
Level	Total_name	sample_size	dm	Model	mdes_effect	achieved_power_at_mdes
Region	Region_100K	1000	1000	Binary (delta; 50_50)	0.02	0.92670
Region	Region_100K	1100	1000	Binary (delta; 50_50)	0.02	0.93960
Region	Region_100K	900	1000	Continuous (rho; Pearson correlation)	0.15	0.84320
Region	Region_100K	1000	1000	Continuous (rho; Pearson correlation)	0.15	0.90055
Region	Region_100K	1100	1000	Continuous (rho; Pearson correlation)	0.15	0.93610
Region	Region_500K	900	1000	Binary (delta; 25_75)	0.02	0.85905
Region	Region_500K	1000	1000	Binary (delta; 25_75)	0.02	0.87150
Region	Region_500K	1100	1000	Binary (delta; 25_75)	0.02	0.88345
Region	Region_500K	900	1000	Binary (delta; 50_50)	0.02	0.89845
Region	Region_500K	1000	1000	Binary (delta; 50_50)	0.02	0.90805
Region	Region_500K	1100	1000	Binary (delta; 50_50)	0.02	0.92155
Region	Region_500K	900	1000	Continuous (rho; Pearson correlation)	0.18	0.94460
Region	Region_500K	1000	1000	Continuous (rho; Pearson correlation)	0.18	0.97180
Region	Region_500K	1100	1000	Continuous (rho; Pearson correlation)	0.15	0.85850

```r
# Export full tables
write_csv(out_cont_CpG,    file.path(out_dir, "EpipwR_full_CpG_cont.csv"))
write_csv(out_bin_CpG,     file.path(out_dir, "EpipwR_full_CpG_binary.csv"))
write_csv(out_cont_Region, file.path(out_dir, "EpipwR_full_Region_cont.csv"))
write_csv(out_bin_Region,  file.path(out_dir, "EpipwR_full_Region_binary.csv"))

# Export combined tables
write_csv(full_results, file.path(out_dir, "EpipwR_full_results_ALL.csv"))
write_csv(mdes_results, file.path(out_dir, "EpipwR_MDES_summary_ALL.csv"))
```

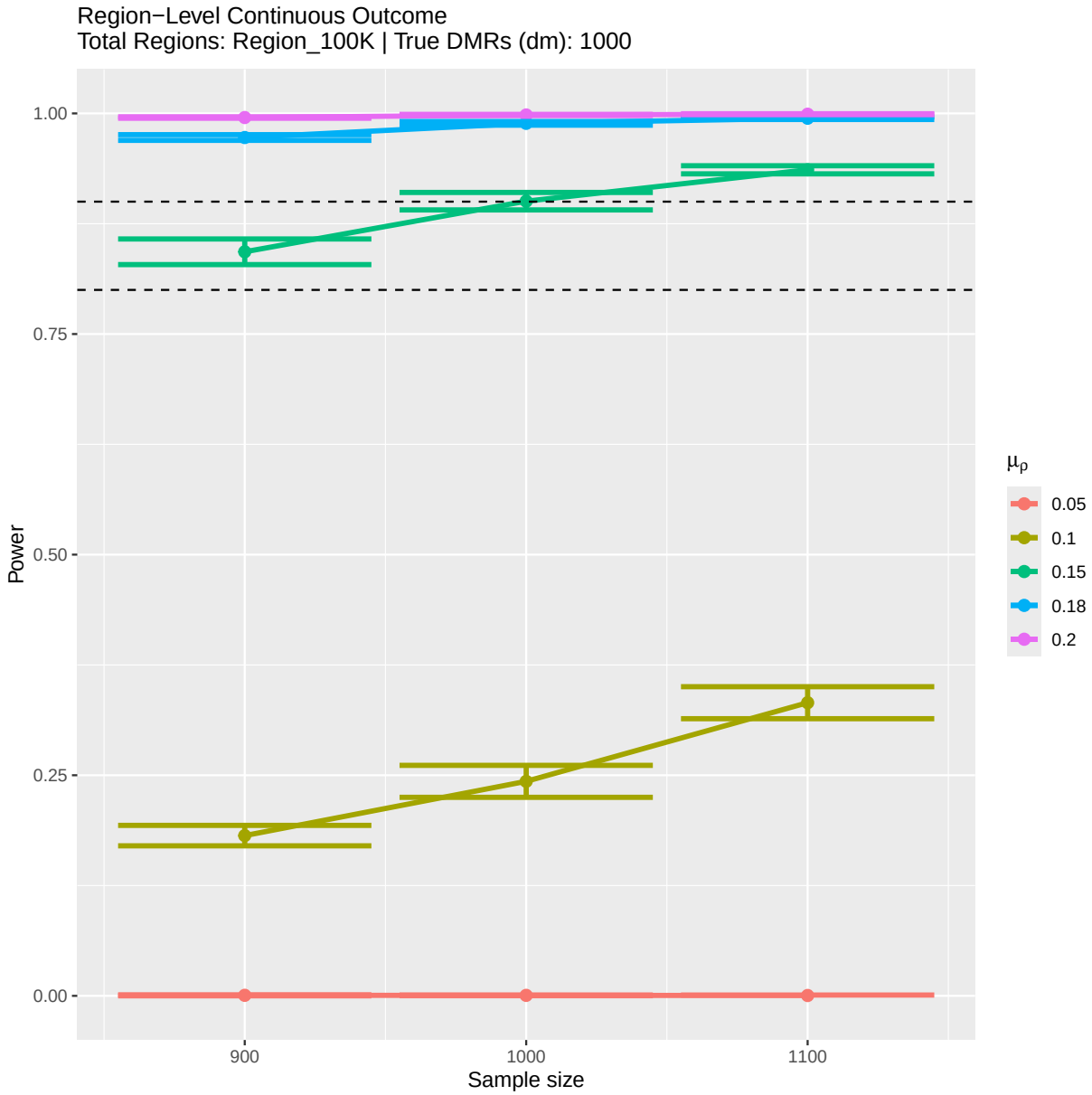
```r
# --- Setup plot directory ---
plot_dir <- file.path(out_dir, "plots")
if (!dir.exists(plot_dir)) dir.create(plot_dir, recursive = TRUE)

# Helper: make a clean, non-clipping title + apply safe theme
make_power_plot <- function(df, title1, subtitle_line, wrap_width = 70,
                            base_text_size = 11) {

  subtitle_wrapped <- paste(strwrap(subtitle_line, width = wrap_width), collapse = "\n")

  EpipwR::EpipwR_plot(df) +
    ggplot2::labs(title = paste(title1, subtitle_wrapped, sep = "\n")) +
    ggplot2::theme(
      # OVERRIDE EpipwR's theme(text=size=18)
      text = ggplot2::element_text(size = base_text_size),

      # Title styling (keep yours, but slightly smaller)
      plot.title  = ggplot2::element_text(size = base_text_size + 1,
                                          margin = ggplot2::margin(b = 8)),

      # Margins (fine)
      plot.margin = ggplot2::margin(t = 12, r = 10, b = 10, l = 10)
    )
}

# ---------------------------
# 1) Region Continuous
```

```r
# ---------------------------
for (tot in unique(out_cont_Region$Total_name)) {
  for (dm in unique(out_cont_Region$dm)) {

    df <- out_cont_Region %>% dplyr::filter(Total_name == tot, dm == dm)

    cat("\n\n## Region Continuous |", tot, "| dm =", dm, "\n\n")

    subtitle <- paste0("Total Regions: ", tot, " | True DMRs (dm): ", dm)

    p <- make_power_plot(
      df = df,
      title1 = "Region-Level Continuous Outcome",
      subtitle_line = subtitle
    )

    print(p)

    ggplot2::ggsave(
      filename = file.path(plot_dir, paste0("Region_continuous_", tot, "_dm", dm, ".pdf")),
      plot = p,
      width = 8, height = 6.5, units = "in"   # height bumped to avoid clipping
    )
  }
}
```

```
##
##
## ## Region Continuous | Region_500K | dm = 1000
```



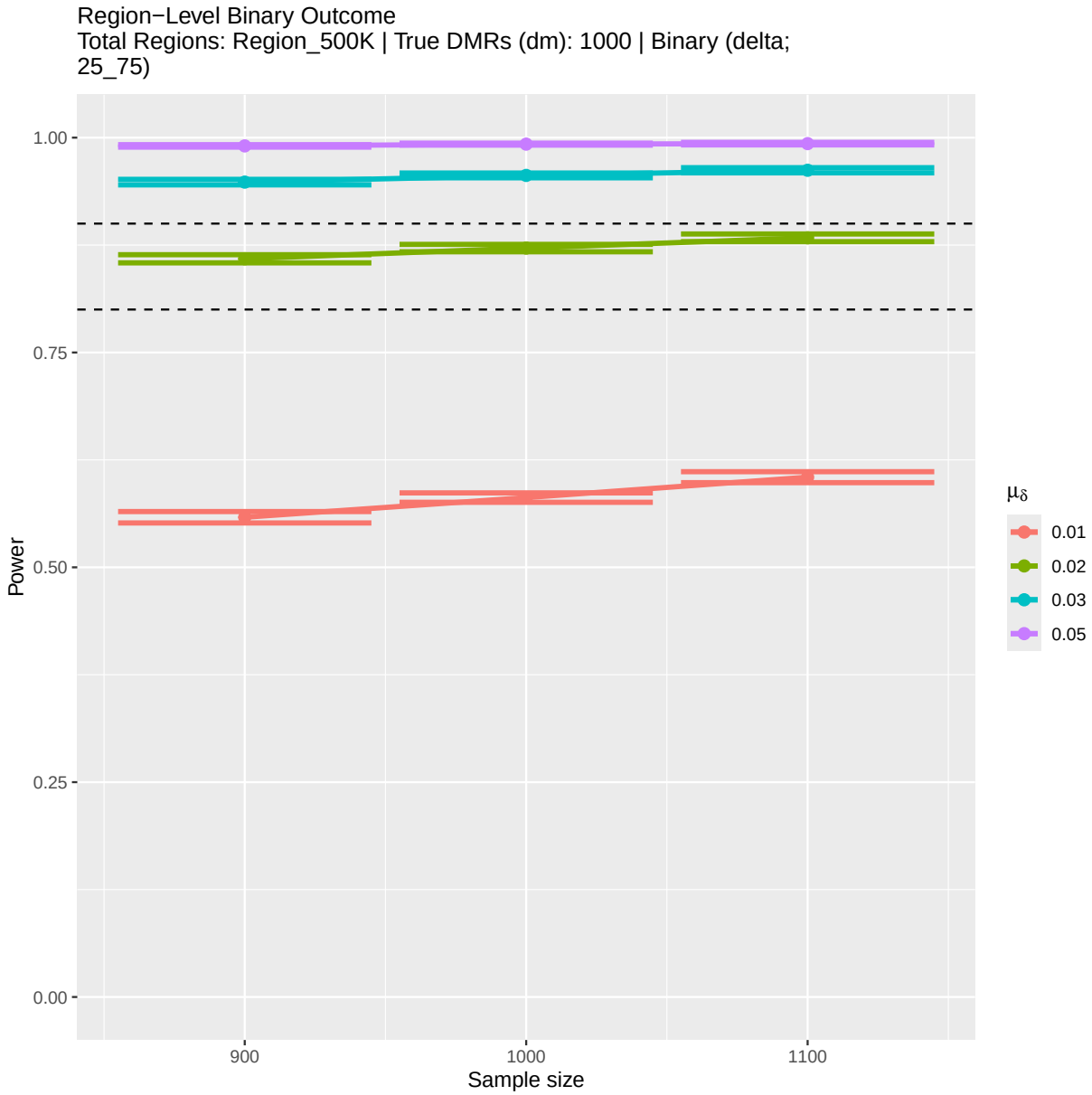

```
##
##
## ## Region Continuous | Region_100K | dm = 1000
```



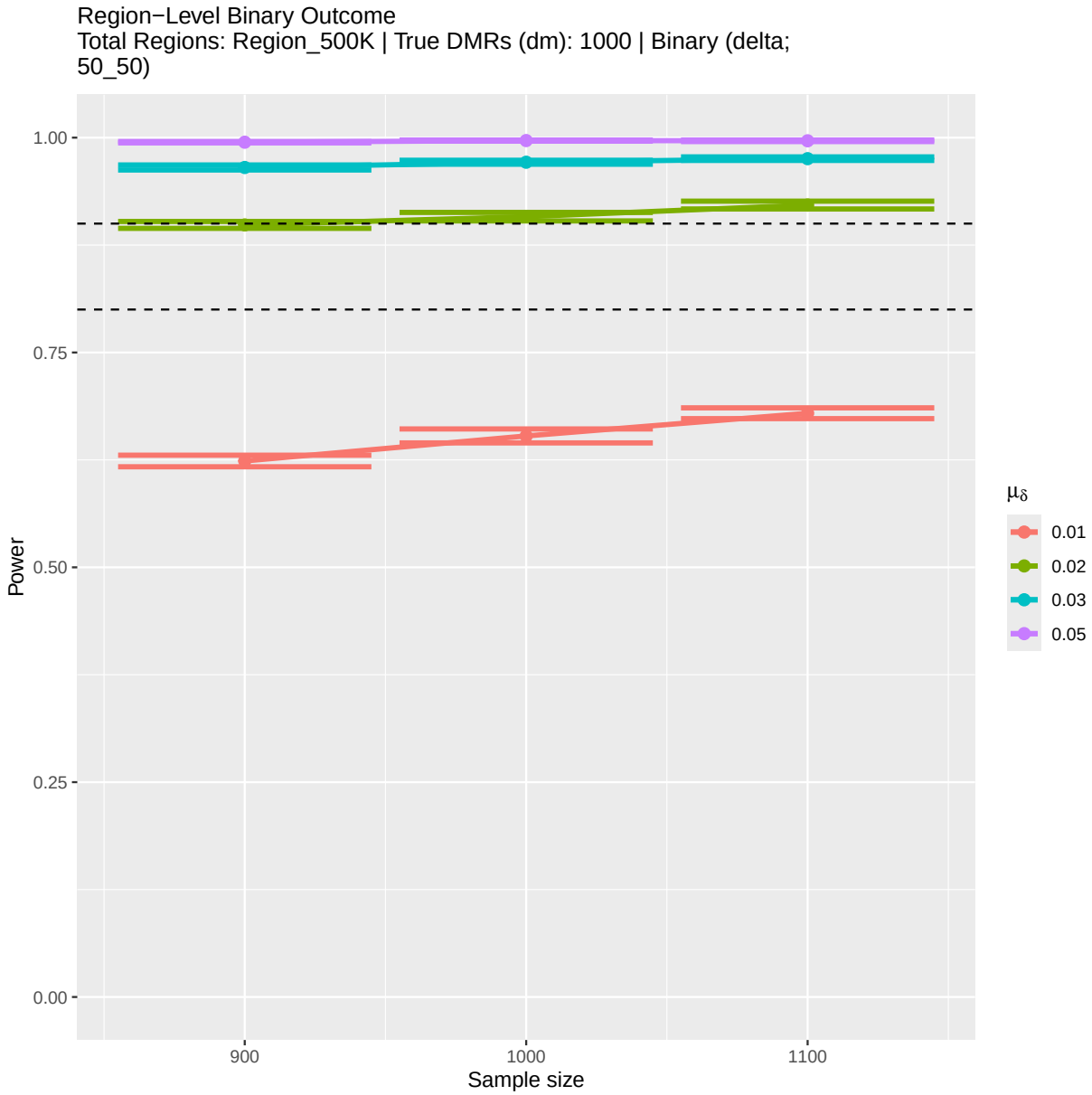

```
# ---------------------------
# 2) Region Binary
# --------------------------
for (tot in unique(out_bin_Region$Total_name)) {
  for (dm in unique(out_bin_Region$dm)) {
    for (model in unique(out_bin_Region$Model)) {

      df <- out_bin_Region %>% dplyr::filter(Total_name == tot, dm == dm, Model == model)

      cat("\n\n## Region Binary |", tot, "| dm =", dm, "|", model, "\n\n")

      subtitle <- paste0("Total Regions: ", tot, " | True DMRs (dm): ", dm, " | ", model)

      p <- make_power_plot(
```

```
        df = df,
        title1 = "Region-Level Binary Outcome",
        subtitle_line = subtitle
      )

      print(p)

      safe_model <- gsub("[^A-Za-z0-9_]+", "_", model)

      ggplot2::ggsave(
        filename = file.path(plot_dir, paste0("Region_binary_", tot, "_dm", dm, "_", safe_model, ".pdf")
        plot = p,
        width = 8, height = 6.5, units = "in"
      )
    }
  }
}
```

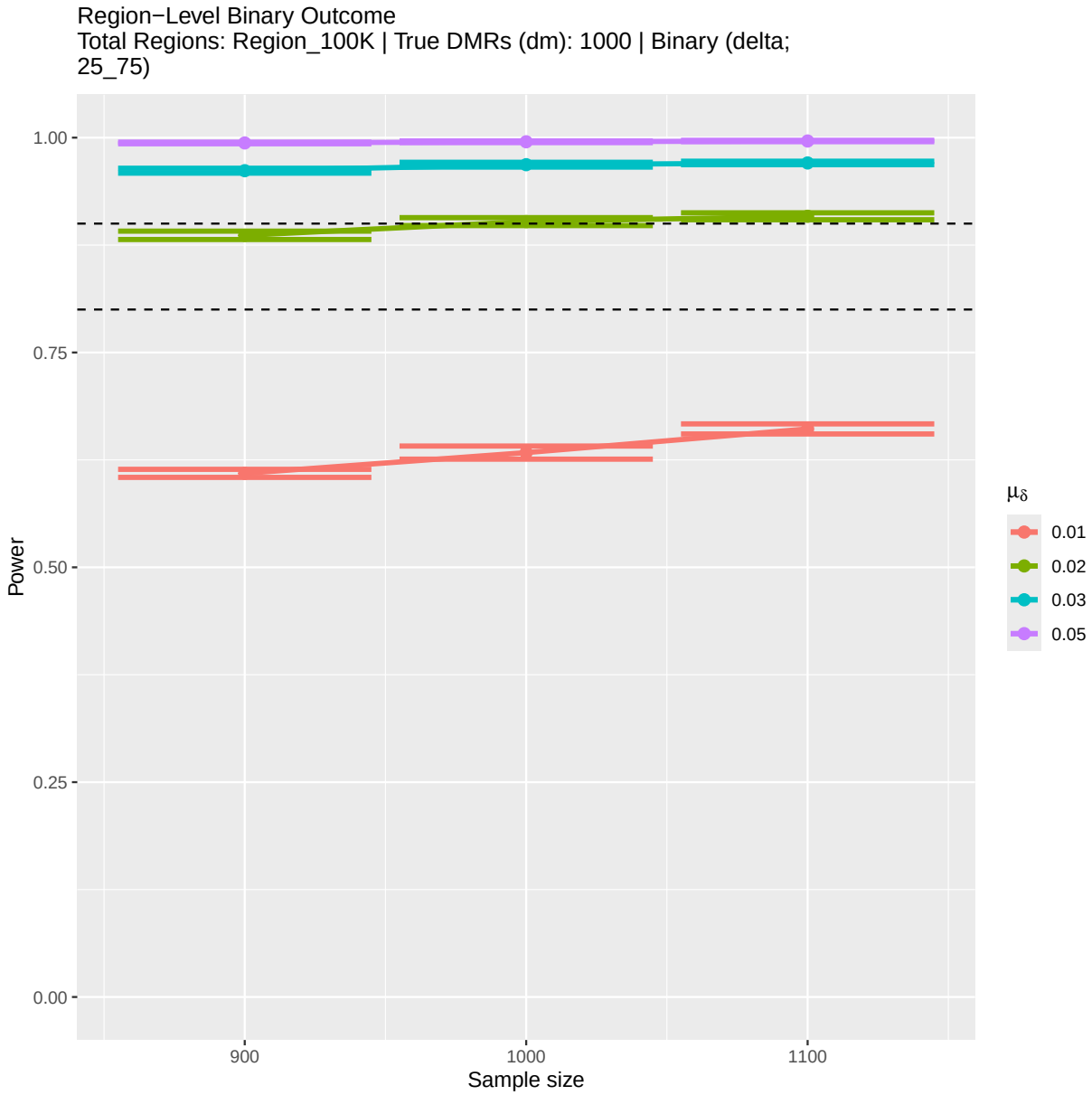
```
##
##
## ## Region Binary | Region_500K | dm = 1000 | Binary (delta; 25_75)
```



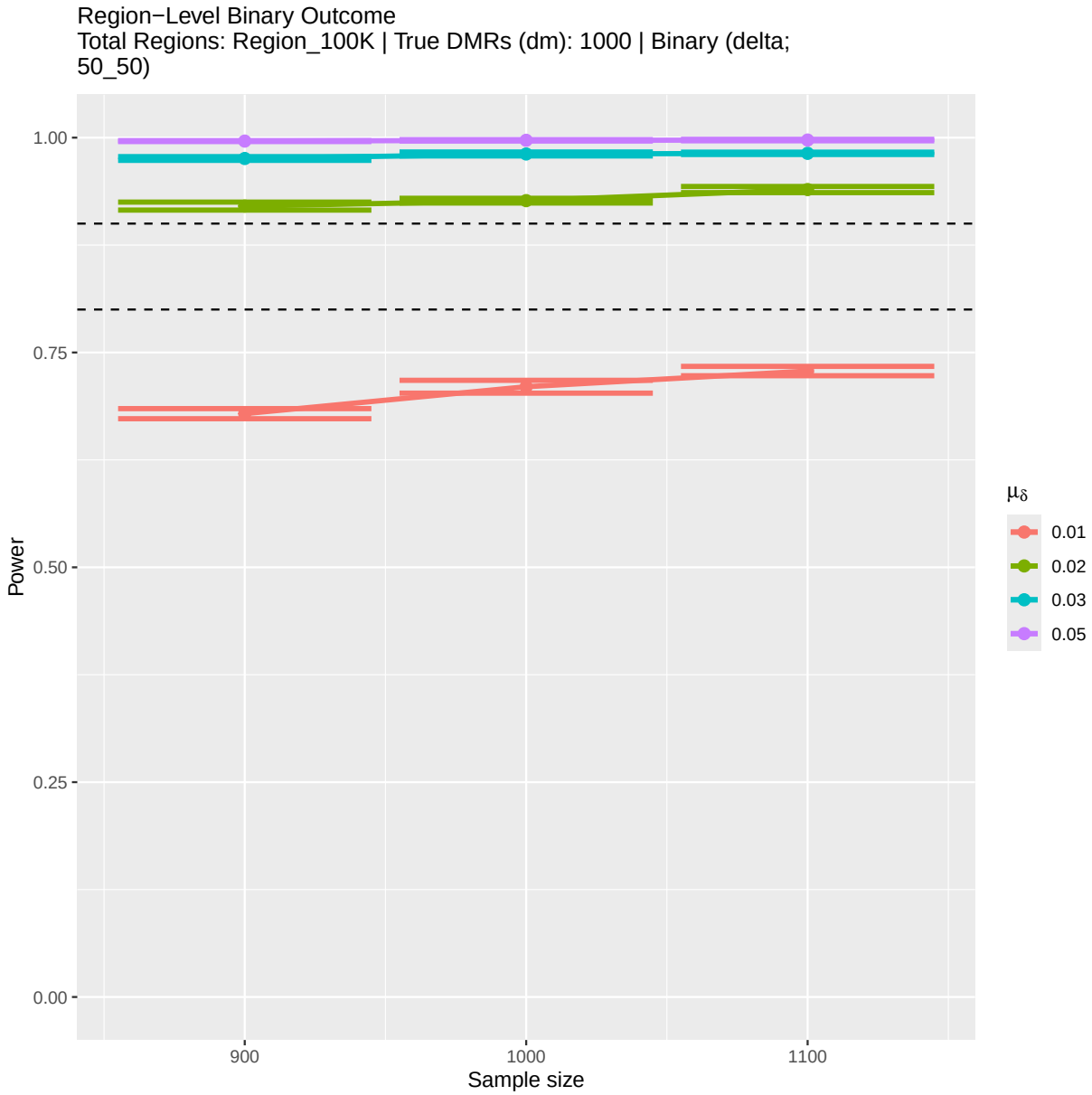

```
##
##
## ## Region Binary | Region_500K | dm = 1000 | Binary (delta; 50_50)
```




```
##
##
## ## Region Binary | Region_100K | dm = 1000 | Binary (delta; 25_75)
```



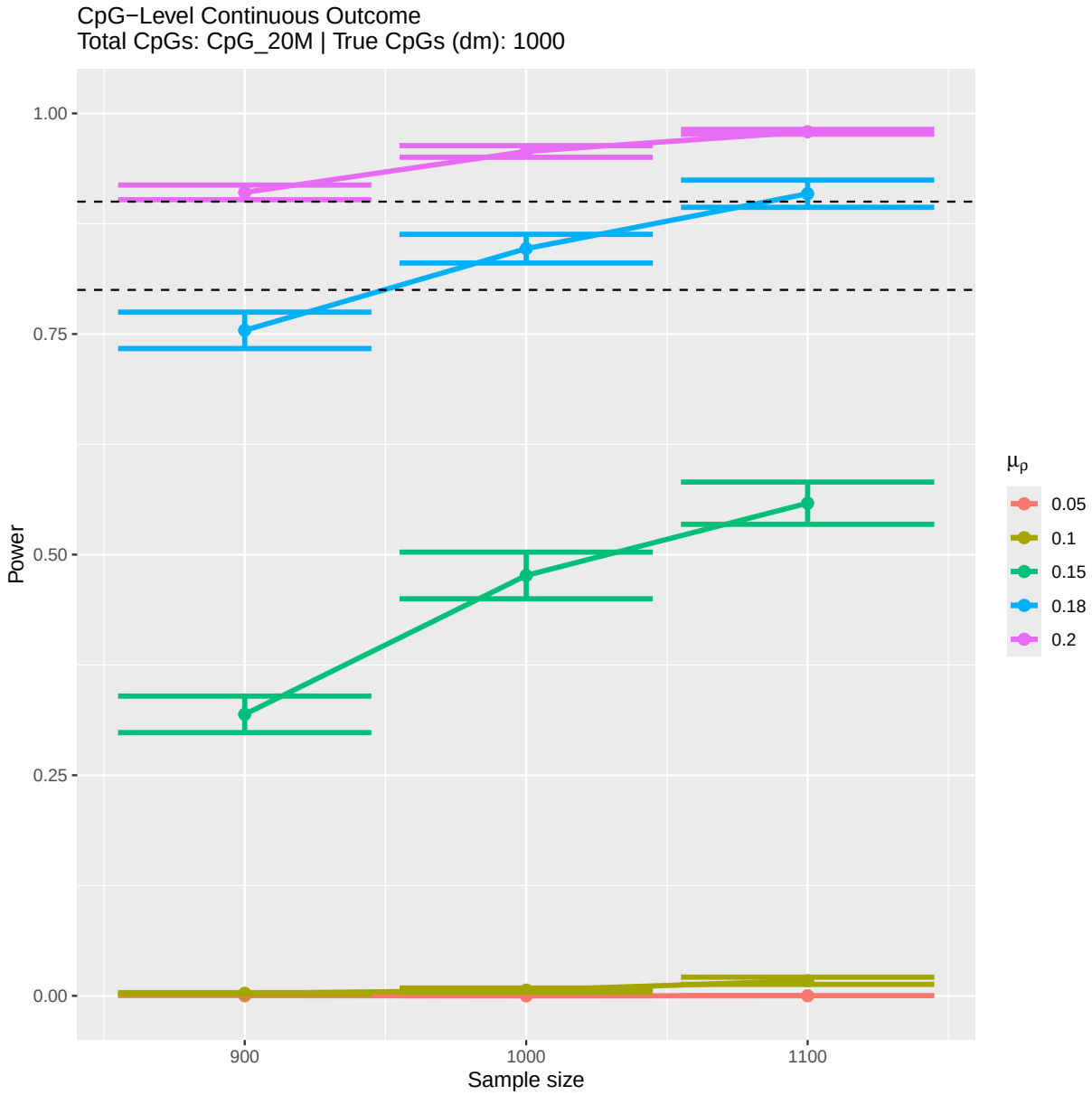

```
##
##
## ## Region Binary | Region_100K | dm = 1000 | Binary (delta; 50_50)
```



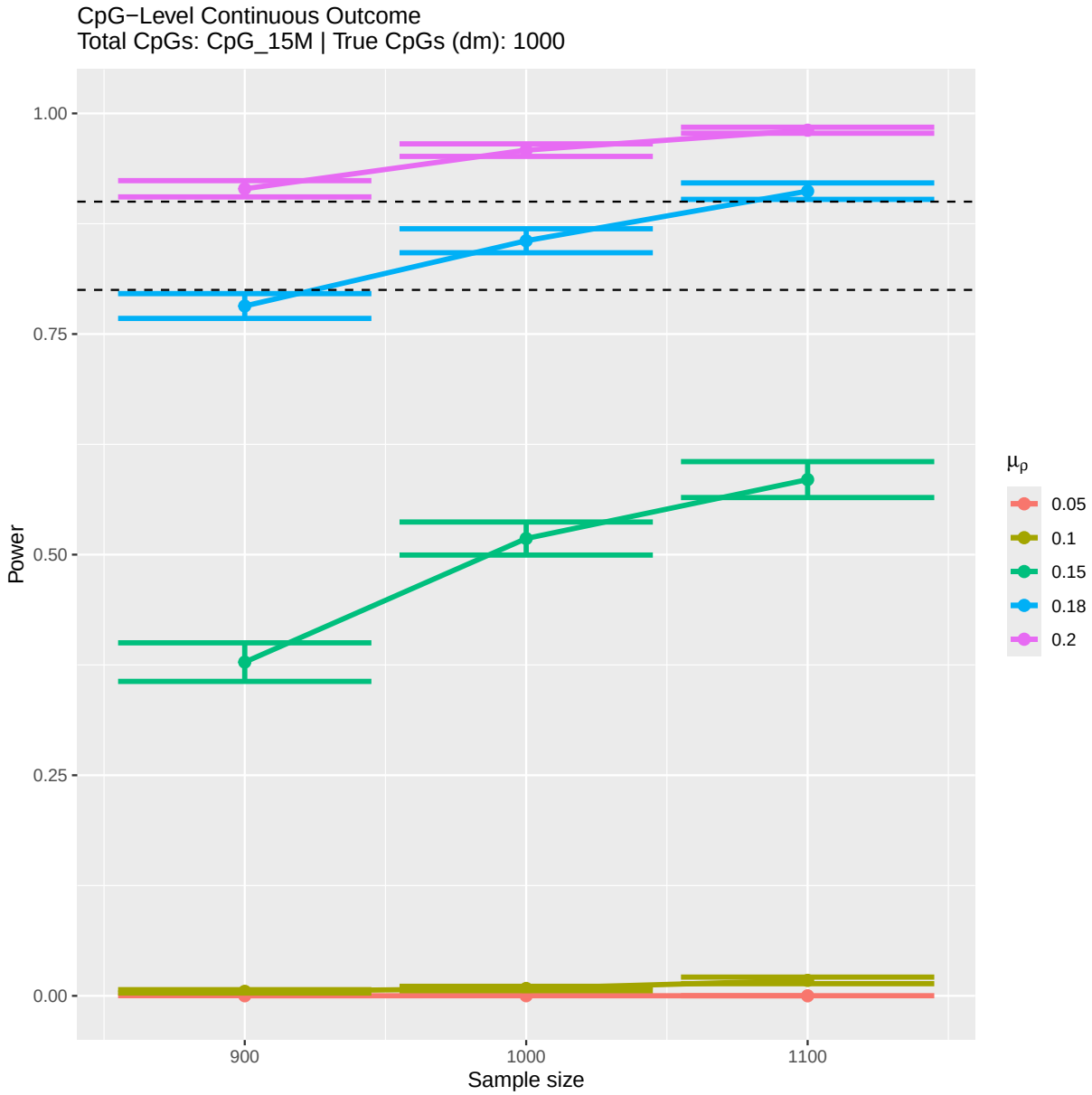

```
# ---------------------------
# 3) CpG Continuous
# ---------------------------
for (tot in unique(out_cont_CpG$Total_name)) {
  for (dm in unique(out_cont_CpG$dm)) {

    df <- out_cont_CpG %>% dplyr::filter(Total_name == tot, dm == dm)

    cat("\n\n## CpG Continuous |", tot, "| dm =", dm, "\n\n")

    subtitle <- paste0("Total CpGs: ", tot, " | True CpGs (dm): ", dm)

    p <- make_power_plot(
      df = df,
```

```r
      title1 = "CpG-Level Continuous Outcome",
      subtitle_line = subtitle
    )

    print(p)

    ggplot2::ggsave(
      filename = file.path(plot_dir, paste0("CpG_continuous_", tot, "_dm", dm, ".pdf")),
      plot = p,
      width = 8, height = 6.5, units = "in"
    )
  }
}
```

```
##
##
## ## CpG Continuous | CpG_20M | dm = 1000
```



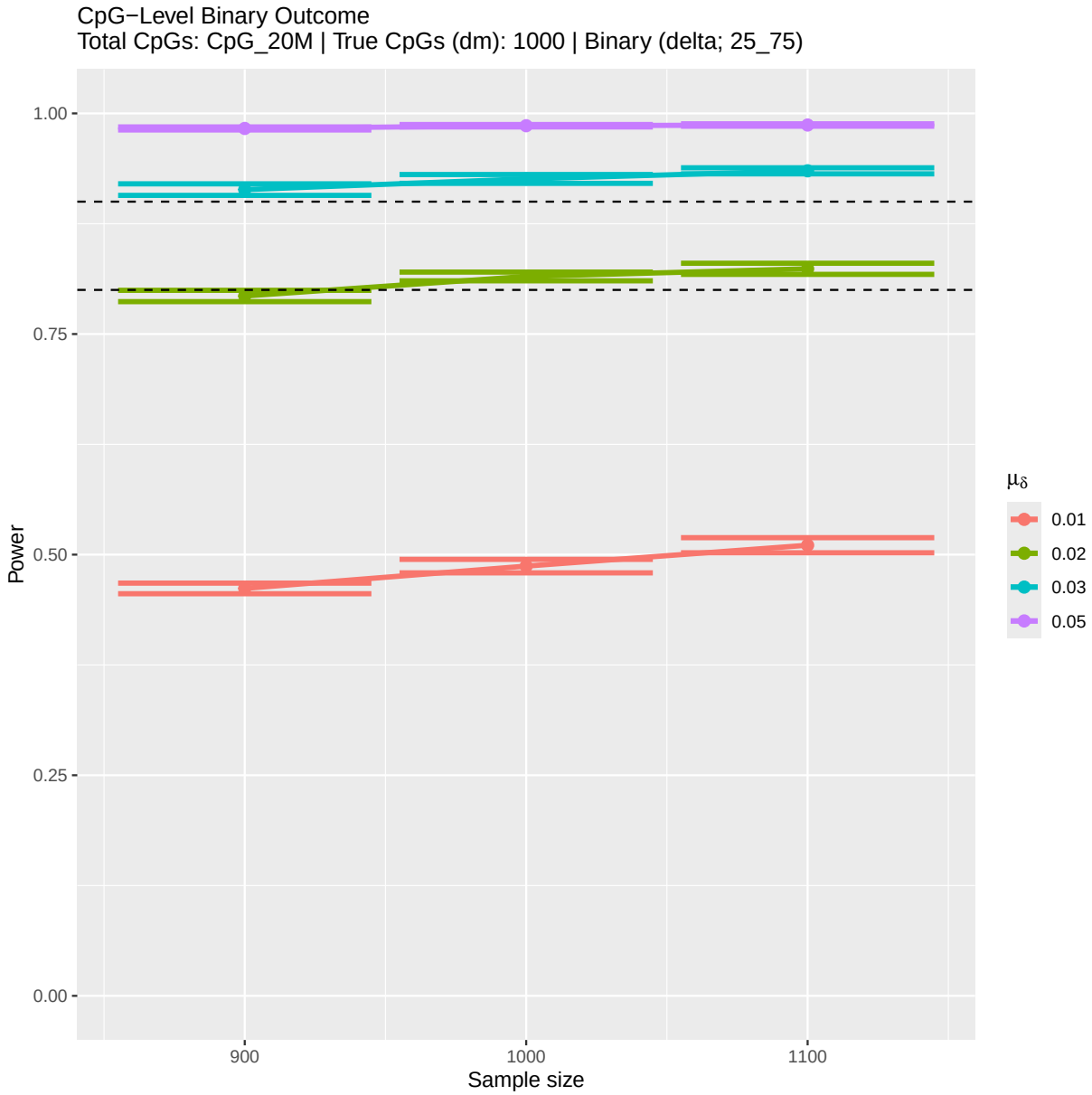

```
##
##
## ## CpG Continuous | CpG_15M | dm = 1000
```



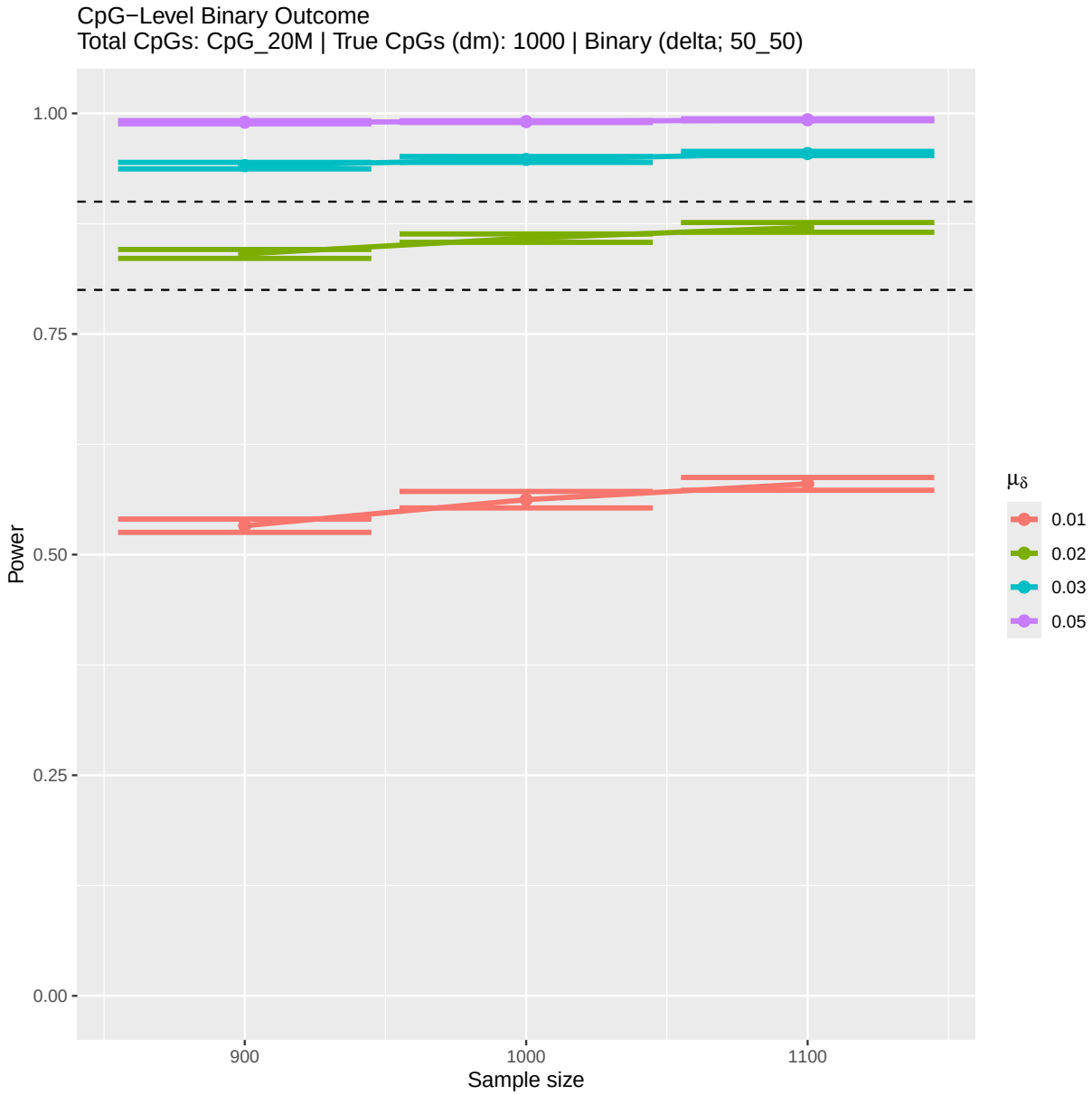

```
# ---------------------------
# 4) CpG Binary
# ---------------------------
for (tot in unique(out_bin_CpG$Total_name)) {
  for (dm in unique(out_bin_CpG$dm)) {
    for (model in unique(out_bin_CpG$Model)) {

      df <- out_bin_CpG %>% dplyr::filter(Total_name == tot, dm == dm, Model == model)

      cat("\n\n## CpG Binary |", tot, "| dm =", dm, "|", model, "\n\n")

      subtitle <- paste0("Total CpGs: ", tot, " | True CpGs (dm): ", dm, " | ", model)

      p <- make_power_plot(
```

```r
        df = df,
        title1 = "CpG-Level Binary Outcome",
        subtitle_line = subtitle
      )

      print(p)

      safe_model <- gsub("[^A-Za-z0-9_]+", "_", model)

      ggplot2::ggsave(
        filename = file.path(plot_dir, paste0("CpG_binary_", tot, "_dm", dm, "_", safe_model, ".pdf")),
        plot = p,
        width = 8, height = 6.5, units = "in"
      )
    }
  }
}
```

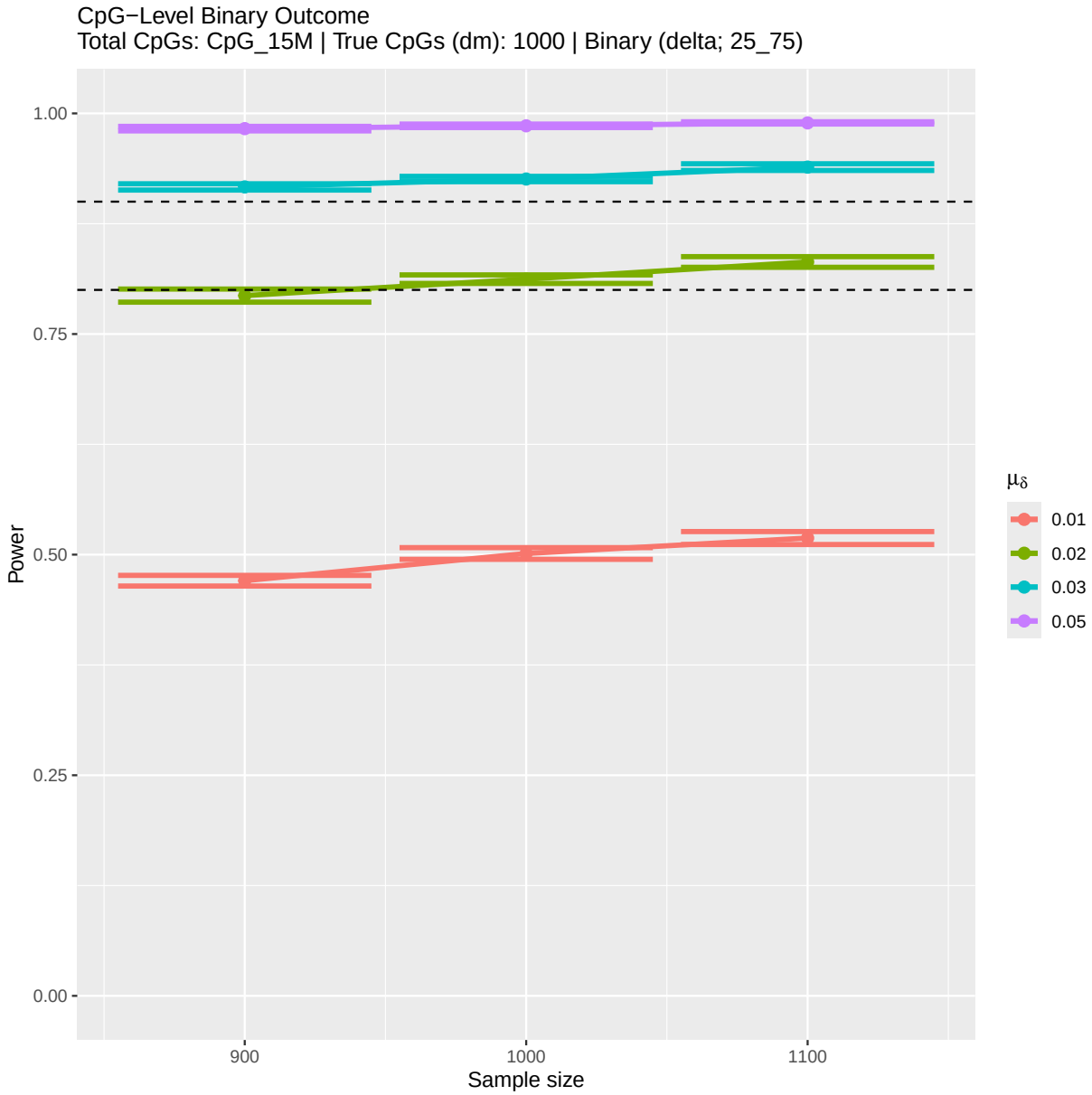
```
##
##
## ## CpG Binary | CpG_20M | dm = 1000 | Binary (delta; 25_75)
```



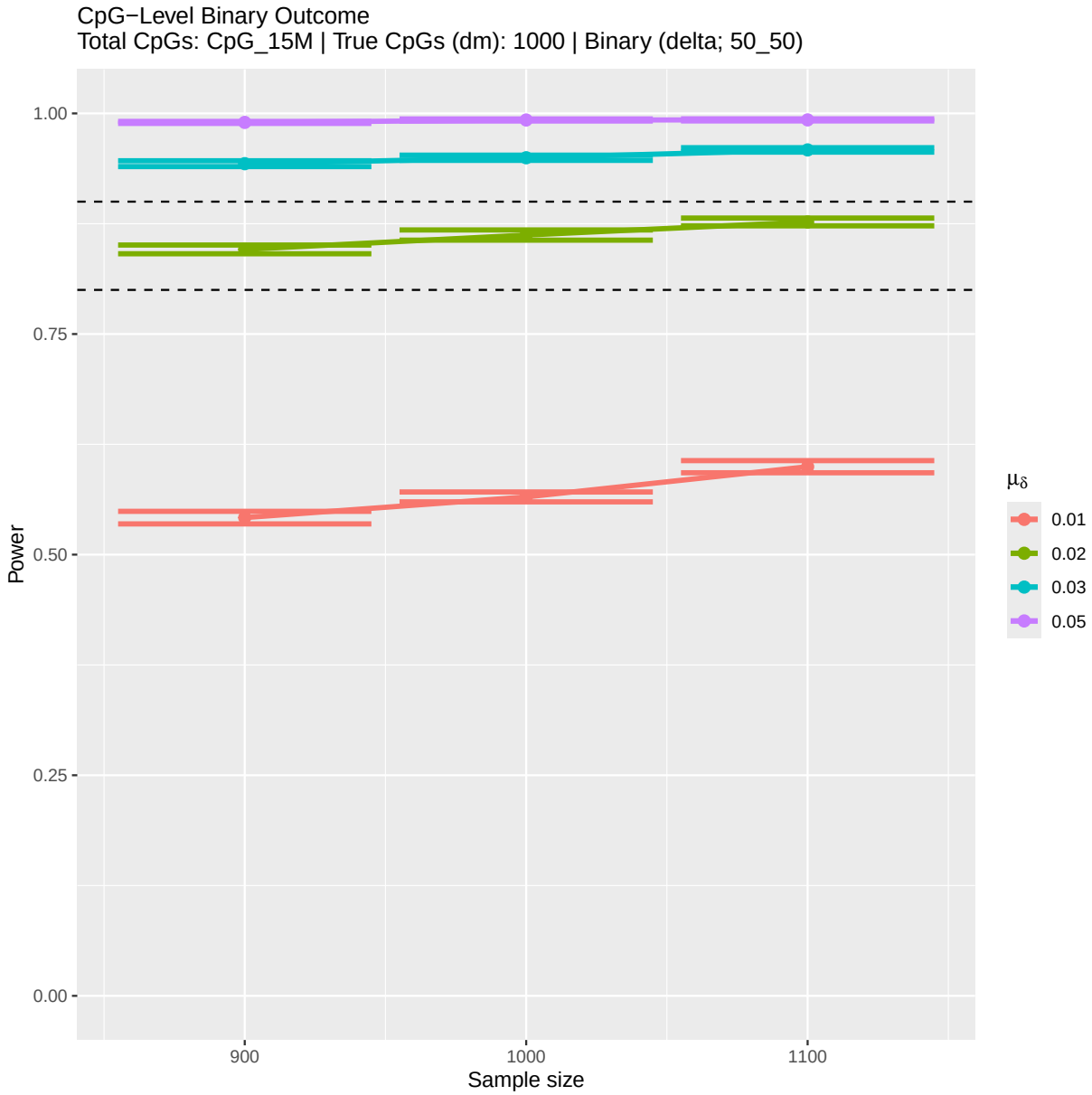

```
##
##
## ## CpG Binary | CpG_20M | dm = 1000 | Binary (delta; 50_50)
```




```
##
##
## ## CpG Binary | CpG_15M | dm = 1000 | Binary (delta; 25_75)
```




```
##
##
## ## CpG Binary | CpG_15M | dm = 1000 | Binary (delta; 50_50)
```




```r
# Load sessioninfo package
library(sessioninfo)

# Display session information
session_info()
```

```
## - Session info ---------------------------------------------------------------
##  setting  value
##  version  R version 4.4.3 (2025-02-28)
##  os       macOS Sequoia 15.6.1
##  system   aarch64, darwin20
##  ui       X11
##  language (EN)
##  collate  en_US.UTF-8
```

```
##  ctype    en_US.UTF-8
##  tz       America/Los_Angeles
##  date     2026-03-03
##  pandoc   3.2 @ /Applications/RStudio.app/Contents/Resources/app/quarto/bin/tools/aarch64/ (via rmarl
##  quarto   1.5.57 @ /Applications/RStudio.app/Contents/Resources/app/quarto/bin/quarto
##
## - Packages ---------------------------------------------------------------
##  package          * version date (UTC) lib source
##  AnnotationDbi      1.66.0  2024-06-16 [1] Bioconductor 3.19 (R 4.4.0)
##  AnnotationHub      3.12.0  2024-04-30 [1] Bioconductor 3.19 (R 4.4.0)
##  Biobase            2.64.0  2024-04-30 [1] Bioconductor 3.19 (R 4.4.0)
##  BiocFileCache      2.12.0  2024-04-30 [1] Bioconductor 3.19 (R 4.4.0)
##  BiocGenerics       0.50.0  2024-04-30 [1] Bioconductor 3.19 (R 4.4.0)
##  BiocManager        1.30.27 2025-11-14 [1] CRAN (R 4.4.3)
##  BiocVersion        3.19.1  2024-04-22 [1] Bioconductor 3.19 (R 4.4.0)
##  Biostrings         2.72.1  2024-06-02 [1] Bioconductor 3.19 (R 4.4.0)
##  bit                4.6.0   2025-03-06 [1] CRAN (R 4.4.1)
##  bit64              4.6.0-1 2025-01-16 [1] CRAN (R 4.4.1)
##  blob               1.3.0   2026-01-14 [1] CRAN (R 4.4.3)
##  cachem             1.1.0   2024-05-16 [1] CRAN (R 4.4.0)
##  cli                3.6.5   2025-04-23 [1] CRAN (R 4.4.1)
##  crayon             1.5.3   2024-06-20 [1] CRAN (R 4.4.0)
##  curl               7.0.0   2025-08-19 [1] CRAN (R 4.4.1)
##  DBI                1.3.0   2026-02-25 [1] CRAN (R 4.4.3)
##  dbplyr             2.5.1   2025-09-10 [1] CRAN (R 4.4.1)
##  digest             0.6.39  2025-11-19 [1] CRAN (R 4.4.3)
##  dplyr            * 1.2.0   2026-02-03 [1] CRAN (R 4.4.3)
##  EpipwR           * 1.3.2   2026-02-28 [1] Github (jbarth216/EpipwR@d280644)
##  EpipwR.data      * 0.99.0  2026-02-28 [1] Github (jbarth216/EpipwR.data@1ec1f5d)
##  evaluate           1.0.5   2025-08-27 [1] CRAN (R 4.4.1)
##  ExperimentHub      2.12.0  2024-04-30 [1] Bioconductor 3.19 (R 4.4.0)
##  farver             2.1.2   2024-05-13 [1] CRAN (R 4.4.0)
##  fastmap            1.2.0   2024-05-15 [1] CRAN (R 4.4.0)
##  filelock           1.0.3   2023-12-11 [1] CRAN (R 4.4.0)
##  generics           0.1.4   2025-05-09 [1] CRAN (R 4.4.1)
##  GenomeInfoDb       1.40.1  2024-06-16 [1] Bioconductor 3.19 (R 4.4.0)
##  GenomeInfoDbData   1.2.12  2024-10-02 [1] Bioconductor
##  ggplot2            4.0.2   2026-02-03 [1] CRAN (R 4.4.3)
##  glue               1.8.0   2024-09-30 [1] CRAN (R 4.4.1)
##  gtable             0.3.6   2024-10-25 [1] CRAN (R 4.4.1)
##  hms                1.1.4   2025-10-17 [1] CRAN (R 4.4.1)
##  htmltools          0.5.9   2025-12-04 [1] CRAN (R 4.4.3)
##  httr               1.4.8   2026-02-13 [1] CRAN (R 4.4.3)
##  IRanges            2.38.1  2024-07-03 [1] Bioconductor 3.19 (R 4.4.1)
##  jsonlite           2.0.0   2025-03-27 [1] CRAN (R 4.4.1)
##  kableExtra       * 1.4.0   2024-01-24 [1] CRAN (R 4.4.0)
##  KEGGREST           1.44.1  2024-06-19 [1] Bioconductor 3.19 (R 4.4.0)
##  knitr              1.51    2025-12-20 [1] CRAN (R 4.4.3)
##  labeling           0.4.3   2023-08-29 [1] CRAN (R 4.4.0)
##  lifecycle          1.0.5   2026-01-08 [1] CRAN (R 4.4.3)
##  magrittr           2.0.4   2025-09-12 [1] CRAN (R 4.4.1)
##  memoise            2.0.1   2021-11-26 [1] CRAN (R 4.4.0)
##  mime               0.13    2025-03-17 [1] CRAN (R 4.4.1)
##  otel               0.2.0   2025-08-29 [1] CRAN (R 4.4.1)
```

```
## pillar            1.11.1  2025-09-17 [1] CRAN (R 4.4.1)
## pkgconfig         2.0.3   2019-09-22 [1] CRAN (R 4.4.0)
## png               0.1-8   2022-11-29 [1] CRAN (R 4.4.1)
## purrr           * 1.2.1   2026-01-09 [1] CRAN (R 4.4.3)
## R6                2.6.1   2025-02-15 [1] CRAN (R 4.4.1)
## ragg              1.5.0   2025-09-02 [1] CRAN (R 4.4.1)
## rappdirs          0.3.4   2026-01-17 [1] CRAN (R 4.4.3)
## RColorBrewer      1.1-3   2022-04-03 [1] CRAN (R 4.4.0)
## readr           * 2.2.0   2026-02-19 [1] CRAN (R 4.4.3)
## rlang             1.1.7   2026-01-09 [1] CRAN (R 4.4.3)
## rmarkdown         2.30    2025-09-28 [1] CRAN (R 4.4.1)
## RSQLite           2.4.6   2026-02-06 [1] CRAN (R 4.4.3)
## rstudioapi        0.18.0  2026-01-16 [1] CRAN (R 4.4.3)
## S4Vectors         0.42.1  2024-07-03 [1] Bioconductor 3.19 (R 4.4.1)
## S7                0.2.1   2025-11-14 [1] CRAN (R 4.4.3)
## scales            1.4.0   2025-04-24 [1] CRAN (R 4.4.1)
## sessioninfo     * 1.2.3   2025-02-05 [1] CRAN (R 4.4.1)
## stringi           1.8.7   2025-03-27 [1] CRAN (R 4.4.1)
## stringr           1.6.0   2025-11-04 [1] CRAN (R 4.4.1)
## svglite           2.2.2   2025-10-21 [1] CRAN (R 4.4.1)
## systemfonts       1.3.1   2025-10-01 [1] CRAN (R 4.4.1)
## textshaping       1.0.4   2025-10-10 [1] CRAN (R 4.4.1)
## tibble          * 3.3.1   2026-01-11 [1] CRAN (R 4.4.3)
## tidyselect        1.2.1   2024-03-11 [1] CRAN (R 4.4.0)
## tzdb              0.5.0   2025-03-15 [1] CRAN (R 4.4.1)
## UCSC.utils        1.0.0   2024-05-06 [1] Bioconductor 3.19 (R 4.4.0)
## vctrs             0.7.1   2026-01-23 [1] CRAN (R 4.4.3)
## viridisLite       0.4.3   2026-02-04 [1] CRAN (R 4.4.3)
## vroom             1.7.0   2026-01-27 [1] CRAN (R 4.4.3)
## withr             3.0.2   2024-10-28 [1] CRAN (R 4.4.1)
## xfun              0.56    2026-01-18 [1] CRAN (R 4.4.3)
## xml2              1.5.2   2026-01-17 [1] CRAN (R 4.4.3)
## XVector           0.44.0  2024-04-30 [1] Bioconductor 3.19 (R 4.4.0)
## yaml              2.3.12  2025-12-10 [1] CRAN (R 4.4.3)
## zlibbioc          1.50.0  2024-04-30 [1] Bioconductor 3.19 (R 4.4.0)
##
## [1] /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/library
## * -- Packages attached to the search path.
##
## ------------------------------------------------------------------------------
```

References

Barth, J., Reynolds, A.W. (2024). EpipwR: Efficient Power Analysis for EWAS with Continuous Outcomes [preprint]. *bioRxiv*, 2024.09.06.611713; doi: https://doi.org/10.1101/2024.09.06.611713

Graw, S., Henn, R., Thompson, J. A., and Koestler, D. C. (2019). pwrEWAS: A user-friendly tool for comprehensive power estimation for epigenome wide association studies (EWAS). *BMC Bioinformatics*, 20(1):218.